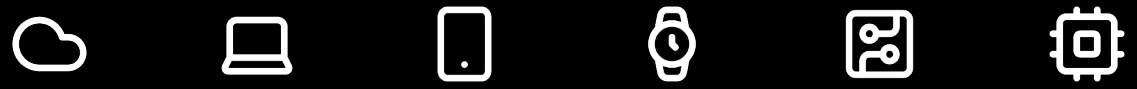


Performance & Energy

Lecture 11: rendering, compilation and energy, from phones to microcontrollers

Dinu-Ștefan Rusu

ORIENTATION

Who's teaching this course

Dinu-Ștefan Rusu

dinu_stefan.rusu@upb.ro
Microsoft Teams course channel
rusudinu.com

45+

Flutter apps shipped

400k

installs across them

Appeared in, spoken at & partnered with

TODAY

What you should leave with

A frame, accounted for	The frame budget as 1 ÷ refresh rate, the three trees, and what each mutation actually costs
JIT and AOT as build modes	Why debug is JIT and release is AOT, on Dart and on ART, and why iOS has no choice
Energy as a design input	Screen, CPU and radio, and why the radio's tail decides your battery rating
TinyGo on bare metal	The same lecture on a microcontroller: sensor → MQTT → your Flutter app, on milliwatts

PART 1 · RENDERING

Where a frame goes

The budget, the three trees, and the operations that exceed it

Two different things we call “performance”

Speed: how fast the app answers.

Startup latency, scroll smoothness, the delay between a tap and something moving. The user feels this directly, in milliseconds.

Efficiency: what the app costs to run.

Memory footprint, install size, and energy. The user feels this indirectly, as a hot phone and a flat battery.

Startup is the metric with a published bar

	What the system already has in memory	Android vitals flags it above
Cold	nothing: process created, runtime initialized, first frame drawn	5 s
Warm	the process, but the screen is rebuilt from scratch	2 s
Hot	process and UI both alive, just brought to the foreground	1.5 s

Measure the one your users are actually hitting. Cold start is what a store reviewer sees; hot start is what a daily user sees ten times a day.

RENDERING

The frame budget is 1 ÷ refresh rate

A frame budget is not a constant. It is one divided by the display's refresh rate, and that rate has been climbing for a decade.

The old “16 ms” is a 60 Hz number. Most phones you will ship to in 2026 run 90 or 120 Hz.

At 120 Hz you have 8.3 ms to run build, layout, paint and rasterization. Miss it and the compositor shows the previous frame again, which is jank.

And jank is variance, not average. A steady 30 fps feels smoother than 55 fps with occasional 100 ms spikes, because the eye tracks rhythm, not throughput.

One frame, in milliseconds

60 Hz	16.7 ms
90 Hz	11.1 ms
120 Hz	**8.3 ms**

Budget for the fastest panel you support, not the slowest. → Lecture 3 introduced this budget; this lecture spends it.

RENDERING

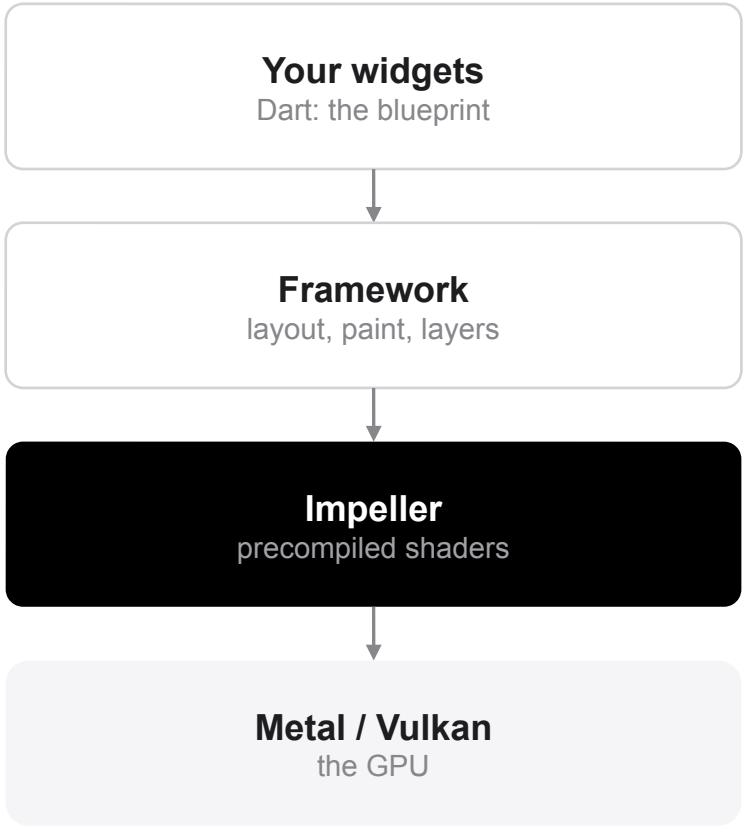
Flutter owns every pixel, and Impeller draws them

Flutter does not drive the platform's widgets. It ships a renderer and paints the whole surface itself. That is why the UI is identical everywhere, and why every performance problem in it is Flutter's, and yours.

The renderer is Impeller, not Skia.

Impeller has been the iOS default since 2023 and the Android default (API 29+, Vulkan) since Flutter 3.29. As of Flutter 3.44 Skia is fully removed on iOS and Android 10+; only very old Android keeps an OpenGL fallback.

The reason it exists: Skia compiled shaders at runtime, the first time an effect appeared on screen. That was the classic “first-run jank”: a blur that stuttered once and never again. Impeller compiles every shader at build time, so the first frame costs the same as the thousandth.



If a tutorial tells you to warm up shaders, it predates Impeller.

Three trees, three very different costs

Widget tree

The blueprint. Immutable configuration objects with no behavior. Allocating one is a bump-pointer write into the young heap: constant time, and genuinely cheap.

Element tree

The mediator. Persists across frames, holds State, and decides, per child, whether the old render object can be kept or must be thrown away.

Render object tree

The machinery. Layout, painting, hit-testing. Expensive to create, expensive to touch, so the framework keeps them alive as long as it possibly can.

Flutter does not rebuild the whole widget tree every frame.

Only subtrees marked dirty rebuild. “Rebuilding widgets is cheap” is a statement about allocation. It is not permission to rebuild the screen for one changed label, because every rebuilt widget still has to be diffed against the element tree.

Reconciliation: runtimeType first, then key

```
// flutter/lib/src/widgets/framework.dart: the whole decision
static bool canUpdate(Widget oldWidget, Widget newWidget) {
  return oldWidget.runtimeType == newWidget.runtimeType
      && oldWidget.key == newWidget.key;
}

// true  → the element is updated in place.
//         State survives. The render object is REUSED.

// false → the old element is deactivated, its render
//         object dropped, a new one inflated and laid out.
```

What this buys you

The element tree walks the new children in order and asks this question once per child: a linear pass, O(N) in the number of children.

Everything you do to make Flutter fast is really about making the answer “true” more often: keep types stable, keep keys stable, and keep the dirty subtree small.

Swapping a Container for a SizedBox in a conditional is a type change, and it silently destroys the state below it.

Same type, same key → the expensive object is kept. Everything else on the next four slides follows from this one line of code.

Keys: identity for things that move

Without a key, children are matched by position. Reorder a list and Flutter hands item 3's state to whatever now sits third: the checkbox stays ticked, the wrong row animates, and the scroll offset jumps.

With a key, Flutter recognizes the move: it relocates the element and its render object instead of rebuilding them, and the state travels with the item.

ValueKey(item.id) for data you own. ObjectKey for identity by instance. UniqueKey only when you genuinely want destruction, since it never matches anything.

GlobalKey is expensive

It can reparent a widget: move a live element under a completely different parent, keeping its state. Three costs come with it:

The element is detached and re-attached, with a lookup in a global registry on top.

Layout and paint are invalidated in both the old location and the new one.

Every InheritedWidget dependency below it has to be resolved again.

Keys are cheap. GlobalKeys are not. Use a GlobalKey when you actually need to reparent or reach a State from outside, not as a convenient handle.

What each operation actually costs

	Cost	Why
Building a widget	constant time	an allocation in the young heap; the generational GC collects it almost for free
Reconciling children	linear, O(N)	one canUpdate call per child; keys keep the answer positive when items move
GlobalKey reparenting	linear search + re-attach	detach, re-attach, invalidate layout and paint in two places
Intrinsic sizing	quadratic, O(N²) when nested	IntrinsicWidth/Height break the single-pass rule: each level walks its subtree again

Layout is one depth-first pass: constraints go down, sizes come up.

A relayout boundary, a subtree whose size cannot depend on its children, stops a dirty flag from propagating upward, which is why a fixed-size box around a busy widget is a real optimization and not a cosmetic one.

OPTIMIZATION

const constructors, and what they save

```
// Rebuilt, re-allocated and re-diffed on every parent build:
Padding(
  padding: EdgeInsets.all(16),
  child: Text('Total'),
)

// Canonicalized at compile time: the SAME instance forever:
const Padding(
  padding: EdgeInsets.all(16),
  child: Text('Total'),
)

# analysis_options.yaml: let the IDE add them for you
linter:
  rules: [prefer_const_constructors]
```

Why it short-circuits the rebuild

Dart canonicalizes const expressions: two identical const widgets are not equal objects, they are the same object.

So when the element compares the new child to the old one, identical(old, new) is true, so it returns immediately without descending. The entire subtree below is skipped.

This is the one optimization that costs nothing but a keyword. Turn the lint on and let the IDE add them.

It only works if every argument is itself a compile-time constant, which is a good reason to hoist styles and paddings into static const fields.

OPTIMIZATION

Lists: build what is on screen, nothing else

```
// Builds all 1,000 children before the first frame is shown.
ListView(children: items.map(RowTile.new).toList())

// Builds only what is visible, plus a small cache extent.
ListView.builder(
  itemCount: items.length,
  itemExtent: 72,        // fixed height: scroll math is O(1)
  itemBuilder: (context, i) => RowTile(
    key: ValueKey(items[i].id),
    item: items[i],
  ),
)

// Heavy parsing never belongs on the UI isolate:
final rows = await compute(parseCsv, bytes);  // → Lecture 3
```

Three things this fixes

Startup: the first frame no longer waits for a thousand build() calls it will never show.

Memory: live elements and render objects stay proportional to the viewport, not to the data.

Scrolling: with itemExtent the framework computes positions arithmetically instead of measuring children.

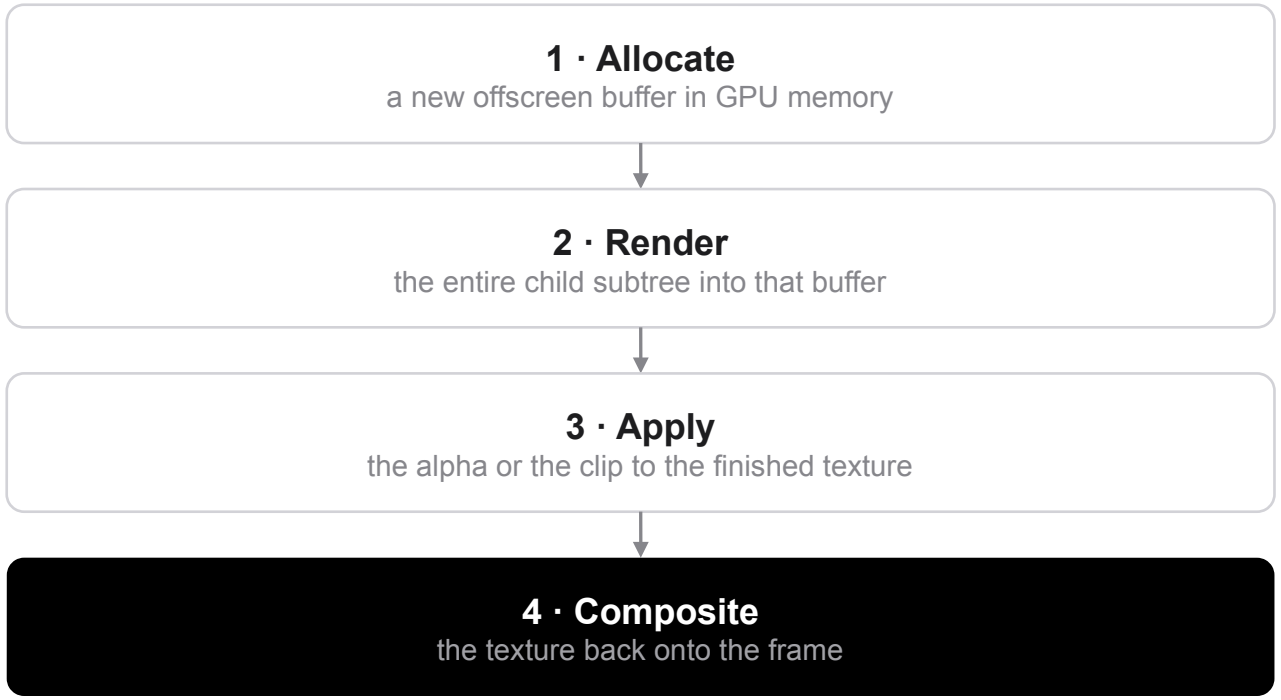
Same idea, more control: SliverList and CustomScrollView, when a list has to share a scroll view with headers.

The bad version is not slow, it is unbounded. It works on the ten rows you tested with and fails on a user's ten thousand.

OPTIMIZATION

Offscreen buffers: when a layer helps and when it hurts

Opacity and ClipRRect can force a saveLayer



Four GPU steps, every frame, for one faded widget. This is the classic green (raster thread) spike.

What to do instead

Animating a fade?

Use AnimatedOpacity or FadeTransition rather than rebuilding an Opacity with a new value each frame. They drive a compositor opacity layer instead of re-diffing the subtree.

Fading a single image?

Image has an opacity: parameter that takes an Animation<double> and blends in the shader, with no offscreen buffer at all.

Hiding something?

Opacity(opacity: 0) still lays out, still paints, still costs. Use Visibility, or simply do not build the widget.

Rounded corners?

A BoxDecoration with a borderRadius paints the shape directly. Reach for ClipRRect only when you must clip real content.

OPTIMIZATION

Find the bottleneck before you fix anything

```
# Debug builds are JIT and assert-heavy. Never quote a debug number.
flutter run --profile        # on a REAL device, not a simulator
# then open DevTools → Performance
```

Blue bar: UI thread	Your Dart is slow: an expensive build(), a setState() too high in the tree, work that belongs in an isolate.
Green bar: raster thread	The GPU is slow: a saveLayer, an over-large image, too many layers, an expensive clip.
Track widget builds	Shows which widgets rebuilt for a frame. If a whole screen lights up for one changed label, that is your bug.
CPU profiler & Memory	Flame chart: look for deep stacks under performLayout or updateChild, which is layout thrashing.

Profile before you change anything. The color of the bar tells you which half of the problem you have.

PART 2 · COMPILATION

JIT, AOT, and the phone in between

Where the machine code comes from, and what that costs in battery

COMPILATION

Three ways to turn your code into instructions

	Interpretation	JIT	AOT
Mechanism	read and execute bytecode instruction by instruction	compile hot paths to machine code while the program runs	compile everything to machine code before the program runs
Startup	instant: nothing to compile	slow at first: you pay the compiler at runtime	instant: the code is already native
Peak speed	lowest	highest: it can optimize on real, observed types	high, but every decision was made without runtime data
Memory	small	largest: a compiler and its buffers live in your process	smallest at runtime; largest on disk

Startup, peak speed and memory trade off: improving one usually degrades another. A phone has to navigate all three at once, on a battery, so nobody ships a pure strategy any more.

COMPILATION

Dart uses both: they are build modes, not camps

```
flutter run                    # DEBUG
#   Dart VM, JIT-compiled. Hot reload works because the VM
#   can swap in newly compiled functions while you run.
#   Asserts on, service protocol open, no optimization.

flutter run --profile          # PROFILE
#   AOT machine code + tracing hooks. This is where you
#   measure. Release numbers, plus DevTools.

flutter build appbundle --release   # RELEASE
#   AOT: Dart compiled straight to ARM machine code.
#   No JIT, no interpreter, no service protocol.
```

JIT for the developer loop, AOT for the user.

Read this carefully

“Dart is JIT” and “Dart is AOT” are both half-true. It is one language with two compilers, and the build mode picks one.

That is the whole trick behind hot reload: development gets the flexibility of a JIT, and users get the startup and the battery of an AOT binary.

Which is also why a stopwatch in debug mode measures nothing. Measure in profile mode.

Two platforms, two different answers

Android: a hybrid, arrived at slowly

Dalvik `Android 2.2–4.4`

trace JIT: compiled while you scrolled, and janked while it did

---

Early ART `Android 5.0–6.0`

pure AOT at install time: fast, but slow updates and huge binaries

---

Modern ART `Android 7.0+`

interpreter + JIT + background AOT when idle and charging

Baseline profiles `Android 9+`

profiles shipped with the app or aggregated from Play: AOT speed on the first launch

iOS: no choice at all

Write XOR execute.

A memory page may be writable or executable, never both at once. It is the defense that stops an attacker who can write bytes from also running them.

A JIT needs exactly that forbidden combination: write machine code into a page, then jump into it. iOS does not grant third-party apps the entitlement, so every app you ship is AOT-compiled native code. WebKit's JavaScript engine is the privileged exception.

Bitcode is gone. Apple deprecated it in Xcode 14 (2022) and no longer accepts bitcode submissions. A build guide that tells you to enable it is four years out of date.

Where cross-platform runtimes stand in 2026

Flutter

Dart AOT-compiles to native ARM for release, so there is no interpreter in production.

Impeller's shaders are compiled at build time too, so the GPU work is decided before you ship.

Debug keeps the JIT, which is what hot reload is.

One pipeline: what you measured in profile mode is what ships.

React Native

Hermes precompiles JavaScript to bytecode at build time, cutting the parse cost at launch.

The new architecture (JSI + Fabric, bridgeless) has been the default since RN 0.76 (Oct 2024).

The old serialized-JSON bridge is gone; JavaScript calls C++ directly.

JS still runs on an engine, so the boundary is cheap now, but not free.

Both ecosystems moved the same work from runtime to build time.

That is the actual trend: not “JIT versus AOT”, but how much can be decided before the user's battery is involved. → Lecture 1 compared these two architectures; this is the compilation half of that story.

PART 3 · ENERGY

Battery as a design constraint

Screen, silicon and radio, and which one dominates the drain

Where the milliamp-hours actually go

The screen

On OLED a black pixel is an unlit pixel, so dark mode is a genuine saving. On LCD the backlight is on regardless and dark mode saves you nothing, so know which panel you are drawing on.

CPU and GPU

Inefficient code raises load, load raises temperature, and the OS throttles the SoC to cool it. Your energy problem returns a few seconds later as a speed problem.

The radio

The most expensive component per byte, and the one that keeps drawing power after your request has finished.

Race to idle.

Modern SoCs are most efficient when they finish a task quickly and go back to sleep. Anything that keeps a core awake, such as a busy animation, a poll loop or a runtime compiler warming up, costs more than the work it performs.

ENERGY

The radio tail: why chatty apps drain batteries

A cellular radio does not switch off when your response arrives. It stays in a high-power connected state for several seconds first, in case more data is coming. The network decides this, not you.

So a request that transfers 200 bytes can keep the radio awake for seconds.

An app that sends one small packet every few seconds never lets the radio step down at all. It pays the tail continuously and shows up in the battery screen next to apps doing a hundred times the work.

What to do about it

Batch. Collect analytics, logs and non-urgent writes and flush them together, not as they happen.

Defer. Hand background work to WorkManager or BGTaskScheduler and let the OS run it when the radio is already awake.

Prefetch on Wi-Fi and while charging, when bytes are cheapest.

Replace polling with push. One held connection or one FCM message beats a timer. → Lecture 8

Send less: compress, page, and do not re-download what you already cached. → Lecture 10

Group your network calls into bursts. The fixed cost is the radio wake-up rather than the bytes, so ten small transfers cost far more than one batched transfer.

PART 4 · EMBEDDED

Four orders of magnitude down

TinyGo on a microcontroller: sensor, radio, and your Flutter app at the other end

The constraint jump: phone → microcontroller

	Phone	Microcontroller
RAM	8–16 GB	**~520 KB SRAM**
Storage	128–512 GB	**~4 MB flash**
Power	~5 W peak, recharged nightly	**milliwatts, sometimes a coin cell for a year**
OS	Android / iOS	**an RTOS, or nothing at all**
Your code	Dart, AOT-compiled	**Go, compiled by TinyGo through LLVM**

This is the far right of the device spectrum from Lecture 1. Every habit from the first three parts of this lecture still applies, and there is no headroom left to absorb a mistake.

EMBEDDED

TinyGo: Go, compiled for bare metal

TinyGo reads ordinary Go with the standard frontend, emits LLVM IR, and lets LLVM produce machine code for Cortex-M, RISC-V, AVR and WebAssembly. It is a different compiler for the same language, not a different language.

What you get

+ Goroutines and channels on a chip with no operating system
+ Binaries measured in tens of kilobytes, not megabytes
+ The machine package: GPIO, ADC, I²C, SPI, UART, PWM, one API per board
+ One language shared with your Go backend → Lecture 5

What you give up

− A reduced runtime: the scheduler is cooperative, the GC is simple

− Limited reflection, so reflection-heavy packages (much of encoding/json) will not build

− A subset of the standard library; no full net stack on most targets

− Some Go packages do not compile, so check before you design around one

---

You keep the language and the concurrency model. You give up the library ecosystem.

Which board to buy

Do not start with the original ESP32

The classic Xtensa ESP32, the one in most tutorials and cheap starter kits, has minimal TinyGo support and no Wi-Fi or Bluetooth from TinyGo. You can blink an LED and very little else.

Buy one of these instead:

ESP32-C3 or ESP32-S3: Wi-Fi from TinyGo via the espradio package, available since TinyGo 0.41 (April 2026). The C3 is a RISC-V part and the cheapest way into a networked project.

Raspberry Pi Pico (RP2040) or Pico 2 (RP2350): first-tier TinyGo support and the best-documented machine package. Take the W variant if you need Wi-Fi.

```
# One target flag is the whole difference.
tinygo flash -target=pico       ./cmd/sensor
tinygo flash -target=esp32c3    ./cmd/sensor

# See what your board actually supports:
tinygo targets
tinygo info pico

# Serial output from the running board:
tinygo monitor
```

Buy a board that TinyGo actually supports. Check tinygo.org/docs/reference/microcontrollers before you order anything.

EMBEDDED

The machine package: a pin, a sensor, a loop

```
// import "machine" and "time" : machine is TinyGo's HAL,
// one API across every supported board.
func main() {
  led := machine.LED
  led.Configure(machine.PinConfig{Mode: machine.PinOutput})

  machine.InitADC()
  sensor := machine.ADC{Pin: machine.ADC0}
  sensor.Configure(machine.ADCConfig{})

  on := false
  for {
    on = !on
    led.Set(on)                // blink: proof of life
    raw := sensor.Get()        // 0 .. 65535, scaled for you
    println("adc:", raw)       // → the serial port
    time.Sleep(2 * time.Second)
  }
}
```

What is different from server Go

There is no main loop provided for you and nothing to return to. If main() exits, the board stops.

machine.LED and machine.ADC0 are aliases the compiler resolves per target, so the same source flashes to a Pico and to an ESP32-C3.

println goes to the serial port, which is why tinygo monitor is your debugger.

For a digital sensor, swap the ADC for machine.I2C0 and a driver from tinygo.org/x/drivers, where there are a few hundred of them.

Goroutines work here: a ticker goroutine sampling into a channel, and main publishing from it, is idiomatic and costs a few hundred bytes of stack.

Publishing a reading over MQTT

```
// 1. join the network (espradio / cyw43439 underneath)
link, _ := netlink.New()
link.NetConnect(&netlink.ConnectParams{
  Ssid: ssid, Passphrase: pass,
})

// 2. connect to the broker
opts := mqtt.NewClientOptions().
  AddBroker("tcp://192.168.1.10:1883").
  SetClientID("mec-sensor-01")
client := mqtt.NewClient(opts)
if t := client.Connect(); t.Wait() && t.Error() != nil {
  panic(t.Error())
}

// 3. publish: topic, QoS, retained, payload
msg := fmt.Sprintf(`{"c":%.1f,"seq":%d}`, tempC, seq)
client.Publish("mec/lab/room1/temp", 0, false, msg)
```

Why MQTT and not HTTP

Packages: tinygo.org/x/drivers/netlink and .../net/mqtt.

A fixed header of two bytes, against several hundred bytes of HTTP headers per request. On a battery, bytes are energy.

The broker fans one reading out to every subscriber, such as a phone, a dashboard or a database, without the device knowing any of them exist.

Retained messages give a late subscriber the last known value immediately, and a last-will message announces that the device has dropped off when it stops answering.

→ Lecture 8 introduced MQTT as one of the four realtime transports. This is the device end of it.

The other end: your Flutter app subscribes

```
// pubspec.yaml:  mqtt_client: ^10.0.0
final client = MqttServerClient('192.168.1.10', 'flutter-hub')
  ..port = 1883
  ..keepAlivePeriod = 30
  ..onDisconnected = _scheduleReconnect;

await client.connect();

// '+' matches one level: every room's temperature.
client.subscribe('mec/lab/+/temp', MqttQos.atLeastOnce);

client.updates!.listen((events) {
  final msg = events.first.payload as MqttPublishMessage;
  final text = MqttPublishPayload.bytesToStringAsString(
      msg.payload.message);
  _controller.add(Reading.fromJson(jsonDecode(text)));
});   // → straight into your BLoC, → Lecture 8
```

The loop the course promised

Sensor → TinyGo → broker → Flutter, with nothing proprietary anywhere in the chain and about forty lines of code at each end.

The phone is the hub: it renders, it stores, it forwards to the cloud. The device only samples and publishes.

Everything you learned about sockets applies unchanged: reconnect with backoff, close in dispose(), and never trust a connection that has gone quiet.

Use wss:// or MQTT over TLS the moment this leaves the lab network. An unauthenticated broker accepts anyone.

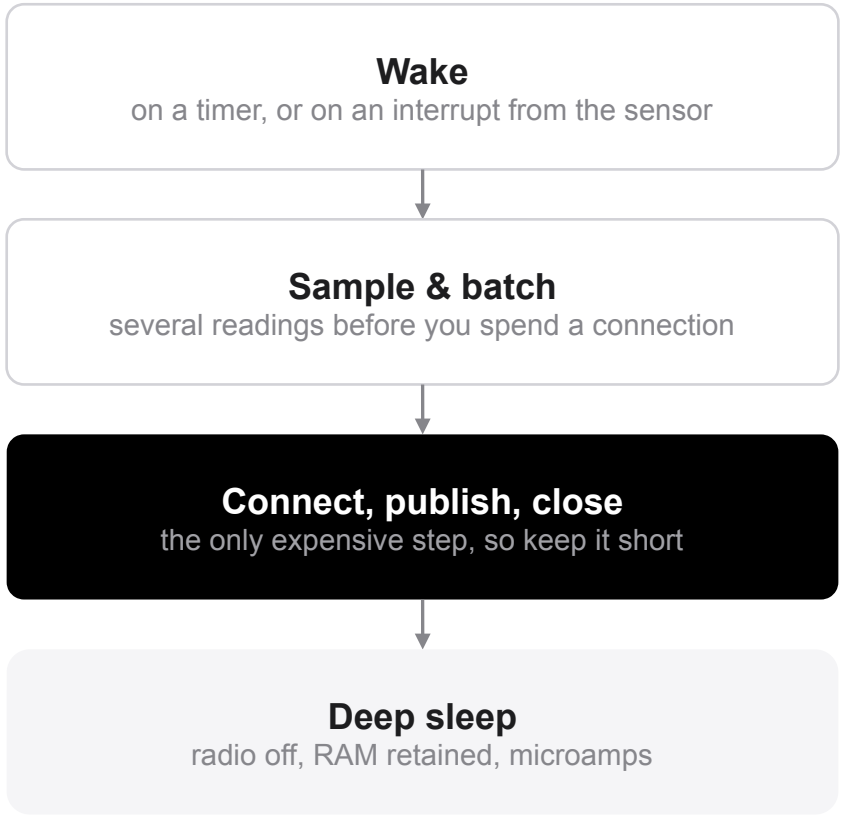
EMBEDDED

Wake, send, sleep: energy at the device level

On an ESP32-class part the datasheet spread between deep sleep and Wi-Fi transmit is roughly four orders of magnitude: microamps against hundreds of milliamps. The radio, not the processor, is the power budget.

So the design question is not “how fast is the code” but “what fraction of the time is the radio on”.

Average current ≈ (active current × active time + sleep current × sleep time) ÷ period. Shorten the active window or lengthen the period, and everything else is noise.



“Stay connected” loses to “wake, send, sleep”.

An MQTT keepalive holds the radio in a powered state forever to save a handshake you only pay once. Unless you need commands pushed to the device within seconds, sleep between publishes and accept the reconnect.

Before next week

Recap

One frame budget = 1 ÷ refresh rate: 8.3 ms at 120 Hz, not 16

Widget cheap, element persistent, render object expensive; canUpdate decides

const, ListView.builder, keys, and no needless saveLayer

JIT and AOT are build modes: debug for you, release for users

Radio dominates energy, on a phone and on a microcontroller alike

This week

Profile your project in --profile mode on a real device; find one red frame and fix it

Turn on prefer_const_constructors and let the IDE do the rest

Order a Pico or an ESP32-C3, not an original ESP32

Flash the blink example and get one reading onto a broker

Read more

docs.flutter.dev/perf

tinygo.org/docs/reference/microcontrollers

pkg.go.dev/tinygo.org/x/drivers

pub.dev/packages/mqtt_client

developer.android.com/topic/performance/vitals